



暨南大学
JINAN UNIVERSITY

本科实验报告

课程名称: 编译原理实验

课程编号: 2425204121

学生姓名: 朱凤丹

学号: 2022103611

学院: 信息科学技术学院

系: 计算机科学系

专业: 计算机科学与技术

指导教师: 王娜

教师单位: 暨南大学

开课时间: 2024 ~ 2025 学年度第二学期

暨南大学教务处

20 年 月 日

暨南大学本科实验报告专用纸

课程名称_____成绩评定_____
实验项目名称_____指导教师_____
实验项目编号_____实验项目类型____实验地点_____学生
姓名_____学号_____
学院_____系_____专业_____
实验时间____年____月____日____午~____月____日____午温度____℃湿度____

一、实验目的和要求

1. 实验目的：理解词法分析在编译程序中的作用；加深对有穷自动机模型的理解；掌握词法分析程序的实现方法和技术。

2. 实验要求：

(1) 待分析的简单的语法

关键字：begin if then while do end

运算符和界符：:= + - * / < <= > >= <> = ;

() #

其他单词是标识符 id 和整型常数 num，通过以下正规式定义：

id=l(l|d)*

num=dd*

空格、注释：在词法分析中要去掉。

(2) 各种单词符号对应的种别编码

单词符号	种别码	单词符号	种别码
begin	1	\0	1000
if	2	(	26
then	3	)	27
while	4	[	28
do	5	]	29
end	6	{	30
int	7	}	31
main	8	,	32
return	12	:	33
cout	13	;	34
l (d) *	10	>	35
:=	18	<	36
dd*	20	>=	37
==	21	<=	38
+	22	!=	40
-	23	"	41
*	24	#	0
/	25	!	-1

程序实现词法分析，从文件 **data.txt** 中读取一段小程序，分解出一个个的单词，其中有关键词，有界符、运算符等等，代码还需实现去掉空格、回车、注释等等情况，最后的输出结果是以单词二元组（单词种别码，单词自身的值）的形式输出。

二、实验内容

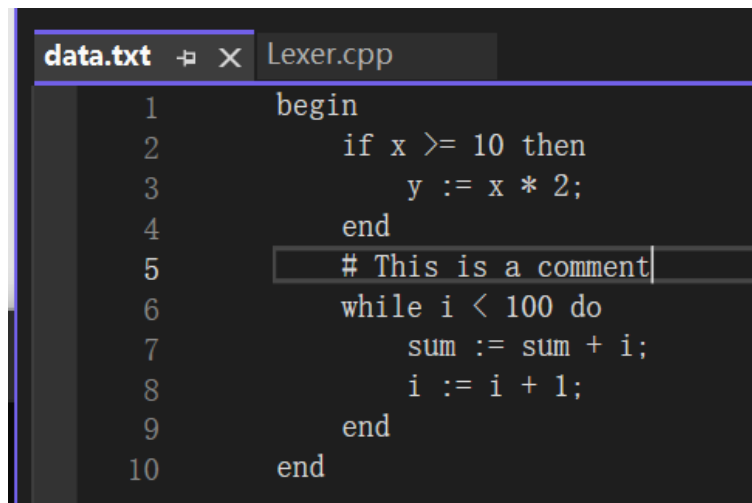
选择部分 C 语言的语法成分，设计其词法分析程序，要求能够识别关键字、运算符、分界符、标识符、常量（至少是整型常量，可以自己扩充识别其他常量）等，并能处理注释、部分复合运算符（如>=等）。

三、主要仪器设备

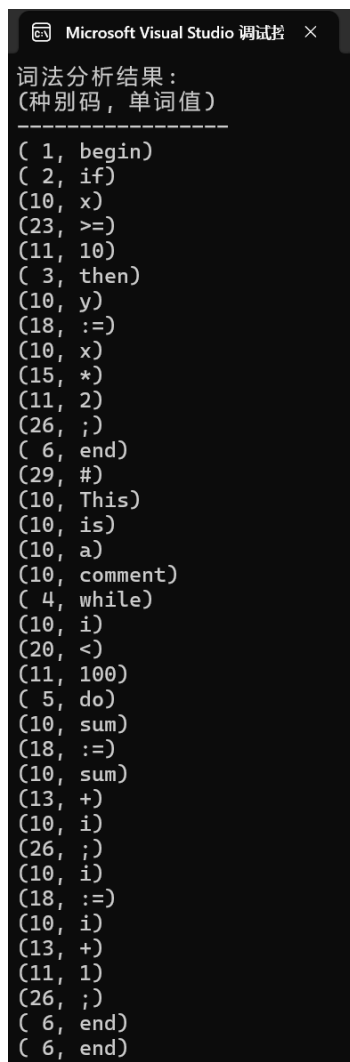
仪器：PC 机

实验环境：Win 7 &&Microsoft Visual Studio 2010

四、测试结果



```
data.txt  Lexер.cpp
1      begin
2          if x >= 10 then
3              y := x * 2;
4          end
5      # This is a comment
6          while i < 100 do
7              sum := sum + i;
8              i := i + 1;
9          end
10     end
```



```
Microsoft Visual Studio 调试器
词法分析结果:
(种别码, 单词值)
-----
( 1, begin)
( 2, if)
(10, x)
(23, >=)
(11, 10)
( 3, then)
(10, y)
(18, :=)
(10, x)
(15, *)
(11, 2)
(26, ;)
( 6, end)
(29, #)
(10, This)
(10, is)
(10, a)
(10, comment)
( 4, while)
(10, i)
(20, <)
(11, 100)
( 5, do)
(10, sum)
(18, :=)
(10, sum)
(13, +)
(10, i)
(26, ;)
(10, i)
(18, :=)
(10, i)
(13, +)
(11, 1)
(26, ;)
( 6, end)
( 6, end)
```

六、附录 （程序代码）

```

#include <stdio.h>
#include <ctype.h>
#include <string.h>

// 单词种别码定义
#define KEY_BEGIN      1
#define KEY_IF         2
#define KEY_THEN       3
#define KEY_WHILE      4
#define KEY_DO          5
#define KEY_END        6
#define ID             10
#define NUM             11
#define OP_PLUS        13
#define OP_MINUS       14
#define OP_MUL          15
#define OP_DIV          16
#define OP_ASSIGN      18
#define OP_LT           20
#define OP_LE           21
#define OP_GT           22
#define OP_GE           23
#define OP_NE           24
#define OP_EQ           25
#define OP_SEMI         26
#define OP_LPAREN       27
#define OP_RPAREN       28
#define OP_HASH         29
#define OP_COMMA        30

typedef struct {
    int type;          // 种别码
    char value[32];    // 单词值
} Token;

// 关键字表
const char* keywords[] = {"begin", "if", "then", "while", "do", "end"};

// 检查是否为关键字，返回种别码
int is_keyword(const char* str) {
    for (int i = 0; i < 6; i++) {
        if (strcmp(str, keywords[i]) == 0) {
            return i + 1; // 种别码 1-6
        }
    }
}

```

```

    }
    return 0;
}

// 获取下一个 token
Token get_token(FILE* fp) {
    static int state = 0;      // 自动机状态
    static char ch;           // 当前字符
    Token token = {0, ""};    // 返回的 token
    char buffer[32] = {0};    // 临时缓冲区
    int index = 0;            // 缓冲区索引

    while (1) {
        switch (state) {
            case 0: // 初始状态
                ch = fgetc(fp);
                if (ch == EOF) {
                    token.type = 0;
                    return token;
                }

                if (isspace(ch)) continue; // 跳过空白

                if (isalpha(ch)) {
                    buffer[index++] = ch;
                    state = 1;
                } else if (isdigit(ch)) {
                    buffer[index++] = ch;
                    state = 2;
                } else if (ch == ':') {
                    buffer[index++] = ch;
                    state = 3;
                } else if (ch == '<') {
                    buffer[index++] = ch;
                    state = 4;
                } else if (ch == '>') {
                    buffer[index++] = ch;
                    state = 5;
                } else if (ch == '/') {
                    state = 6;
                } else {
                    // 处理单字符运算符
                    buffer[index++] = ch;
                    buffer[index] = '\0';
                }
            }
        }
    }
}

```

```

        switch (ch) {
            case '+': token.type = OP_PLUS; break;
            case '-': token.type = OP_MINUS; break;
            case '*': token.type = OP_MUL; break;
            case '=': token.type = OP_EQ; break;
            case ';': token.type = OP_SEMI; break;
            case '(': token.type = OP_LPAREN; break;
            case ')': token.type = OP_RPAREN; break;
            case '#': token.type = OP_HASH; break;
            case ',': token.type = OP_COMMA; break;
            default: token.type = -1; // 错误
        }
        strcpy(token.value, buffer);
        state = 0;
        return token;
    }
    break;

case 1: // 标识符或关键字
    ch = fgetc(fp);
    if (isalnum(ch)) {
        buffer[index++] = ch;
    } else {
        ungetc(ch, fp); // 回退字符
        buffer[index] = '\0';
        token.type = is_keyword(buffer) ? is_keyword(buffer) : ID;
        strcpy(token.value, buffer);
        state = 0;
        return token;
    }
    break;

case 2: // 数字
    ch = fgetc(fp);
    if (isdigit(ch)) {
        buffer[index++] = ch;
    } else {
        ungetc(ch, fp);
        buffer[index] = '\0';
        token.type = NUM;
        strcpy(token.value, buffer);
        state = 0;
        return token;
    }
}

```

```
break;
```

```
case 3: // 冒号（可能形成:=）
```

```
    ch = fgetc(fp);
    if (ch == '=') {
        strcpy(token.value, ":=");
        token.type = OP_ASSIGN;
    } else {
        ungetc(ch, fp);
        strcpy(token.value, ":");
        token.type = -1; // 错误
    }
    state = 0;
    return token;
```

```
case 4: // <（可能形成<=或<>）
```

```
    ch = fgetc(fp);
    if (ch == '=') {
        strcpy(token.value, "<=");
        token.type = OP_LE;
    } else if (ch == '>') {
        strcpy(token.value, "<>");
        token.type = OP_NE;
    } else {
        ungetc(ch, fp);
        strcpy(token.value, "<");
        token.type = OP_LT;
    }
    state = 0;
    return token;
```

```
case 5: // >（可能形成>=）
```

```
    ch = fgetc(fp);
    if (ch == '=') {
        strcpy(token.value, ">=");
        token.type = OP_GE;
    } else {
        ungetc(ch, fp);
        strcpy(token.value, ">");
        token.type = OP_GT;
    }
    state = 0;
    return token;
```

```

case 6: // (可能形成注释)
    ch = fgetc(fp);
    if (ch == '/') { // 单行注释
        while ((ch = fgetc(fp)) != '\n' && ch != EOF);
        state = 0;
    } else if (ch == '*') { // 多行注释
        int valid = 0;
        while ((ch = fgetc(fp)) != EOF) {
            if (ch == '*') {
                if ((ch = fgetc(fp)) == '/') {
                    valid = 1;
                    break;
                }
            }
        }
        if (!valid) token.type = -2; // 注释未闭合
        state = 0;
    } else { // 普通除号
        ungetc(ch, fp);
        strcpy(token.value, "/");
        token.type = OP_DIV;
        state = 0;
        return token;
    }
    break;
}
}
}
}

```

```

int main() {
    FILE* fp = fopen("data.txt", "r");
    if (!fp) {
        perror("无法打开文件");
        return 1;
    }

    Token token;
    printf("词法分析结果:\n");
    printf("(种别码, 单词值)\n");
    printf("-----\n");

    while (1) {
        token = get_token(fp);
        if (token.type == 0) break; // 文件结束
    }
}

```

```
    if (token.type < 0) {  
        printf("错误: 无效符号 %s\n", token.value);  
        continue;  
    }  
    printf("(%2d, %s)\n", token.type, token.value);  
}  
  
fclose(fp);  
return 0;
```